2019 IEEE International Conference on Big Data (Big Data)

A Fast Exact Viewshed Algorithm on GPU

Faisal F. Qarah
Department of Computer Science and Engineering
University of South Florida
Tampa, Florida, USA
faisalq@mail.usf.edu

Yi-Cheng Tu
Department of Computer Science and Engineering
University of South Florida
Tampa, Florida, USA
tuy@mail.usf.edu

***Abstract*—We present a parallel GPU algorithm for computing viewshed using the radial-sweep approach, and we are considering the case of in-memory execution. We compared our algorithm with other three viewshed algorithms: sequential R3, sequential radial-sweep, and parallel GPU-based R3. In our experiment, we used a grid of size (10801 x 10801), and our algorithm finished execution in 1.7 seconds and was more than 3.5x faster than GPU-based R3, over 6200x times faster than the standard sequential R3 algorithm, and 1627x times faster than the sequential CPU-based radial-sweep. Viewshed problem or visibility computing is concerned with finding all points or cells that are visible for a giving cell that is called the observer. Processing such a problem for a large grid is very time-consuming using CPU, mainly when producing results with the highest possible accuracy. Thus, in our work, we are proposing a new parallel algorithm for solving the viewshed problem. Our algorithm is based on the radial-sweep algorithm, and it is designed to be used on the GPU. The algorithm design has overcome many challenges to enable the parallelization of the radial-sweep algorithm. Challenges such as minimizing the space required by intermediate results used by the algorithm, efficiently sorting the intermediate results, utilizing pre-allocated static data structures instead of dynamic data structures to be more suitable for GPUs, and dividing the problem into chunks to be processed in parallel while maintaining the accuracy of the results to be equivalent to its CPU sequential version.**

***Index Terms*—viewshed computation, parallel algorithm, computing visibility, GPU**

I. Introduction

In this paper, we consider computing visibility on a large elevation grid (10,000 x 10,000 cells) of a specific point v located within the map using GPU. Computing visibility or viewshed yields all points in the map that are visible to point v. Visibility computing has a wide range of applications in Geographic Information Systems (GIS), 2D and 3D game development, and computer graphics. It helps to solve problems such as forming communication networks for telephone, radio, and television [1] [2], military facilities and natural resources site hiding [3]. In the past, elevation grids and maps used to be of low-resolution with small sizes, and with that, it used to be a very time-consuming task to compute the viewshed. Now with current advances in creating large maps with higher resolution, viewshed computing became more time consuming, and CPU trends still cannot keep up with processing large maps even if it fits in memory which forced GIS developers to give up on implementing exact-viewshed algorithms and focus more on approximate algorithms that give an acceptable amount of accuracy within a reasonable time. Thus, a different approach has to be taken to solve the "exact" viewshed problem efficiently, especially when processing large maps and grids. We choose to design our new algorithm based on the fastest sequential exact-viewshed algorithm, which called radial-sweep or plane-sweep [4] to be used on the GPU. Designing a GPU-based radial-sweep algorithm was a challenging task to do, but the results are very satisfying in terms of speed. Radial-sweep algorithm falls under the class of exact viewshed algorithms that tend to process every cell in the elevation-grid. In general, viewshed processing tends to be a compute-intensive task, which makes it suitable for the GPU due to its capability in exploiting data-level parallelism and its SIMD nature along with its high memory bandwidth that surpasses CPU's memory bandwidth.

A. Terrain Representation

In this work we will focus on viewshed algorithms that are based on regular square grids (RSG) [1] as its terrain data format due to the wide use for terrain and maps representation where a map is represented in the form of a grid consists of a group of equally sized cells. Such representation allows to easily compute the visibility of each cell individually. The other type of terrains, which are triangulated irregular networks (TIN) whereas the name indicates the surface is composed of irregular triangles of different sizes [1]. This type of representation will not be considered in our work. However, both types of representation follow the same mathematical model called the digital elevation model (DEM) that contains the elevation measures for all points in the terrain in a 2.5D manner [1] [5].

B. Problem Definition

Assume we have a grid of n x n equal-sized cells, and for each cell, there is an elevation value h_i similar to [4] [5] [6] [7], and for a giving cell that is called the observer v located within the terrain T, we want to know all visible cells to v where all the cells are in T. These cells are called target cells c. In order for c to be visible to v, the elevation angle $E\theta$ has to be higher than or equal to all elevation angles for the cells located between c and v [3] [4] [7]. To reduce computation a little bit, we can consider the elevation slope obtained from equation (1) instead of going all the way to compute the elevation angle similar to [6].

978-1-7281-0858-2/19/$31.00 ©2019 IEEE

$$C_{ElevationSlope} = \frac{(h_i - h_{poi})}{d_{vc}} \quad (1)$$

Where h_i is the height of cell c, h_{poi} is the height of our point-of-interest v, and d_{vc} is the distance between v and c.

$$E\theta = \arctan\left(C_{ElevationSlope}\right) \quad (2)$$

II. RELATED WORK

The viewshed computation problem was the focus of many researchers due to its importance in many applications. [3] has presented two standard algorithms for viewshed computation R3 and R2. R3 originally presented in [8] and it is considered to be the standard solution for viewshed problem. R3 also is considered to be an exact-viewshed algorithm where it computes the visibility for every cell in the grid by launching a line-of-sight (LOS) starting from cell v all the way to target cell c. For every cell in-between v and c, the LOS computes these cells elevation angle as it goes through them and see if any of them would obstruct the target cell from being seen by the observer cell v. For an n x n grid, the time complexity for R3 is O(n^3) where we have n^2 cells in the grid, and all of them will be checked independently for visibility, and when checking every target cell, the line-of-sight will go through at most O(n) cells that lie between v and whatever target cell. However, R3 algorithm tends to suffer from being impractical, and it may take several hours to solve on CPU even with today's hardware to process a grid of size (10,000 x 10,000) as our experiment show. The other option is R2 algorithm, which is an approximation for solving the problem of viewshed where it launches several LOS from cell v to only border cells in the grid. For all cells in between v and any border cell, the LOS that is the closest to the center of the cell in the middle will conduct visibility check on that cell using the information such as the highest elevation angle so far that it collected when traversing the previous cells located between v and the middle cell. See Fig. 1. R2 algorithm still gives an acceptable degree of accuracy [3] [9], but it differs based on data used. To the best of our knowledge, there is still no affirmative answer on how much accuracy we lose when using R2.

Another exact-viewshed algorithm proposed by [4] called radial-sweep or plane-sweep viewshed algorithm that has an accuracy equivalent to R3 [7], but it follows a different approach than R2 and R3. The main idea of the algorithm is that we assume there is a line from the observer cell v and it rotates around v sweeping all cells. If the sweeping line crossed a cell, it computes its visibility to v. The time complexity for the algorithm is O(n^2 log n) where it sweeps all cells once, and for each cell it will make what is called an event thus we have O(n^2) events total. Those events will be handled temporary in a binary tree called status-tree, and the cost for inserting an event to the status-tree and then deleting it is O(log n), so the overall time complexity for the algorithm is O(n^2 log n). We will describe the algorithm in more detail in the next section.

All these algorithms are designed for sequential programming model and based on the assumption that all data fit in memory. Many researchers designed variations of these algorithms for many different cases. [6] presented an I/O efficient variation of the sequential radial-sweep algorithm where it first generates and sorts the events-list in a streaming manner and then it recursively divides the events-list until it size is suitable to fit in memory along with the status-tree where they assume that it has to be in memory all the time at the start of visibility computing phase. [10] proposed an I/O efficient algorithm for R2 where the grid is divided into blocks, and whenever there is a need for a cell to be processed located within a particular block, that block gets fetched to main memory. They adopted an LRU policy for management blocks in memory, and the results show significant improvement compared to prior work. Even though processing large grids using R2 and radial-sweep took up to hours as reported in [6] [10] due to the algorithms sequential nature and no parallelism has been exploited. [7] was the first to target GPU in processing viewshed. They presented a GPU parallel version of R3 that is an I/O efficient considering that GPUs global memory was much smaller at that time. [11] have implemented R2 on GPU, and they showed the speed up of GPU-based R2 over the sequential R2 on CPU. [9] also designed R2 algorithm on GPU. However, they considered the case when data do not fit in memory. Thus they proposed an I/O efficient R2 algorithm on GPU where the grid is divided into horizontal segments that could fit in GPU global memory, and segments are streamed from disk to pined buffer in CPU memory and then to GPU global memory. This reduces the data transfer cost from CPU's to GPU's memory. [5] proposed a parallel radial-sweep algorithm on the CPU. However, they implemented the same technique used in R2 to find the cells within each sector, which transforms the radial-sweep algorithm from being an "exact" viewshed computing algorithm to an "approximate" algorithm, which explains the high efficiency of the execution time reported in their paper. In their algorithm, in order to parallelize radial-sweep, a preliminary step is needed to enable parallelism. When dividing the grid into sectors with equal azimuth angle, they needed a way to find all the cells that reside within each sector. They had to throw several rays equal to the number of border cells in every sector from observer cell to perimeter/border cells, and for each ray, they apply the DDA line drawing algorithm [12] to identify all cells by computing cells x-value and y-value that intersects with lunched ray. Our algorithm, however, does not require any preliminary computations and preserves the accuracy of computing visibility equivalent to its sequential version.

III. RADIAL-SWEEP ALGORITHM DESIGN ON GPU

In this section, we will go through a brief introduction on the GPU architecture and its most prominent features. Also, the sequential algorithm design will be explained in detail in this section, along with the proposed GPU version of it.

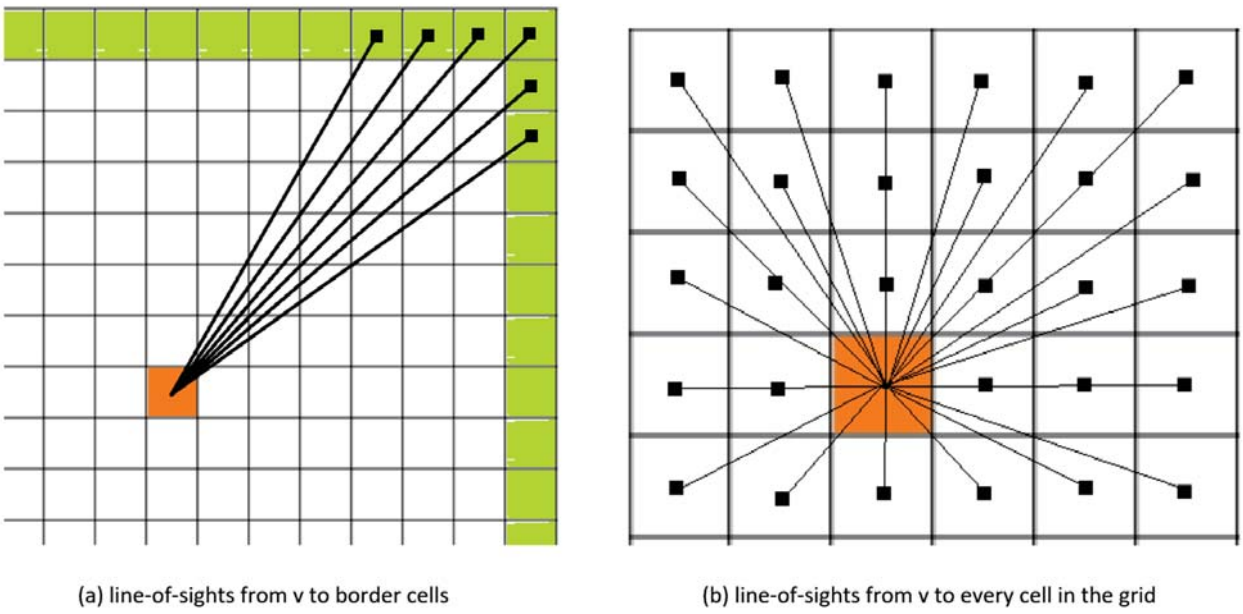


Fig. 1. Illustration for viewshed algorithms R2 and R3. (a) In R2, line-of-sight rays start from observer cell v to border cells. Total number of LOS equal to the number of border cells. (b) In R3, LOS rays are launched from the observer cell to all cells in the grid. The total number of LOS equal to the total number of cells in the grid.

A. GPU Architecture

Before discussing the design of the viewshed algorithm on GPU, it is necessary first to briefly introduce the main components of the GPU and GPGPU environment. Specifically NVidia GPU devices along with CUDA programming model. In general, NVidia GPUs are composed of streaming multiprocessors (SMs), and each SM contains over a hundred computing core. For example, Volta architecture contains 80 streaming multiprocessors, and each one is composed of 64 computing cores. This large number of cores work together in a SIMD manner where all cores will run the same instructions in any given time over different data. Each computing core runs a single thread at a time, and threads are grouped into blocks. The number of threads in each block is between 32 and 1024 threads per block, and we can run $2^{32} - 1$ blocks per kernel. Threads within a block can utilize a unique type of memory called shared memory, and it can be used for communication between threads and for storing temporary results that are frequently needed due to the high bandwidth shared memory have. Another type of memory GPU has is the register file; it is used by threads to store temporary variables and data, which defines the amount of workload a thread can do. Surprisingly, the capacity of the register file exceeds shared memory, which leads to more workload for threads. See Fig. 2. The last type of memory used by GPUs that we want to mention is global memory. It serves as RAM for the GPU but with higher bandwidth and faster access time than CPU's RAM. Global memory access time, however, is slower than shared memory. Thus it is preferable to exploit shared memory whenever possible to improve the algorithm performance.

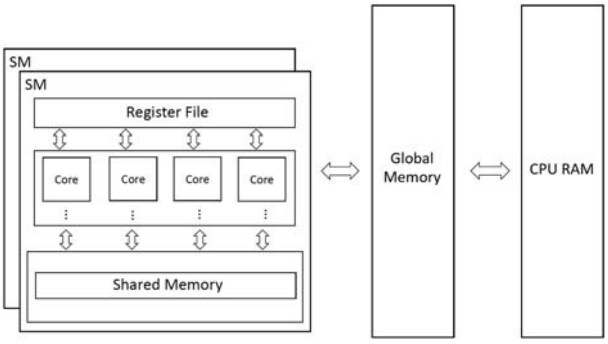


Fig. 2. NVidia GPUs architecture

B. Radial-Sweep Algorithm

Radial-sweep algorithm proposed by [4] to compute viewshed with an accuracy equivalent to R3 but with less execution time and much better time complexity. Assume there is a line-of-sight that rotates around the observer cell v sweeping all cells around it. See Fig. 3. Whenever the LOS comes across any cell's center, it starts computing that cell visibility to v. The rotating line completes a full round when rotating for 360°. The algorithm mainly uses two data structures status-tree and event-list. Status-tree or status-structure temporarily stores information about all cells that are currently intersecting with the rotating line. The status-tree is a special augmented

balanced binary search tree where its leaves contain information about the current intersected cells where the cell that is closest to the observer v is in the leftmost leaf, and the farthest is located in the rightmost leaf. The internal nodes contain only information about the elevation angles only for the cells are currently in the status-tree which is computed using equations 1 and 2. Note, however, that any internal node only keeps the highest elevation-angle value presented only by its children nodes. For example, the root node contains the highest elevation-angle provided by all leaf nodes. This helps in determining the visibility of any cell that the rotating line intersected its center quickly by checking the left child of every node it crosses when searching the target cell. See Fig. 4. Overall the time complexity for inserting, searching, or deleting operations occurs in O(log n) time, where the status-tree will contain at most n cells at any giving time. Every cell in the grid (except the observer) will have precisely three events: insert, delete, and inquiry. See Fig. 5. Insert event means adding a cell to the status-tree, delete means removing a cell from the status-tree, and inquiry means checking the visibility of that cell by searching the status-tree. An event occurs when the sweeping line intersects a certain angle or a location within a cell. For example, the insert event for a cell that is located in the first quadrant will be triggered when the sweeping-line makes contact with it for the first time, in other words when the line intersects the bottom-right corner of that cell, and for delete event is when the sweeping line intersects the top-left corner of that cell see figure for all insert and deletes angles for cells in different quadrants. The search or inquiry event is triggered when the sweeping line intersects the center of any cell. In general, we first generate all events for all the cells in the grid based on their angle relative to the observer cell, and then we store all generated events in what is called the event-list. After that, events in the event-list are sorted based on the angle theta. If there are two events with the same theta value, we then put the event with the smallest distance first. Note that events angles are different from elevation angles. Events angles can be obtained using equations (3) and (4). Also, the event-angle values for insert and delete events are different from inquiry event. This means the parameters `target_x` and `target_y` in equation (3) will need some proper adjustment based on the cell location as shown in Fig. 5.

$$D_x = target_x - poi_x, Dy = target_y - poi_y \tag{3}$$

$$E\theta = \arctan(\frac{Dy}{Dx}) \tag{4}$$

C. Radial-Sweep Algorithm on GPU

Our proposed algorithm, which is the GPU version of the original sequential radial-sweep, follows the same guidelines. However, we divide our algorithm into four main steps:

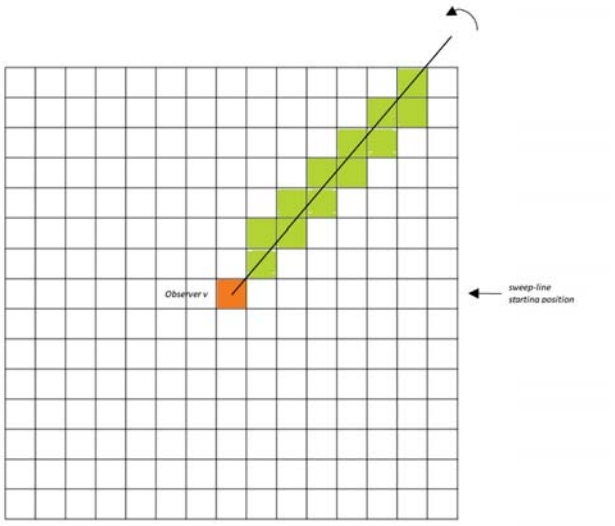


Fig. 3. Illustration of the sweeping-line and cells that are currently intersecting with it.

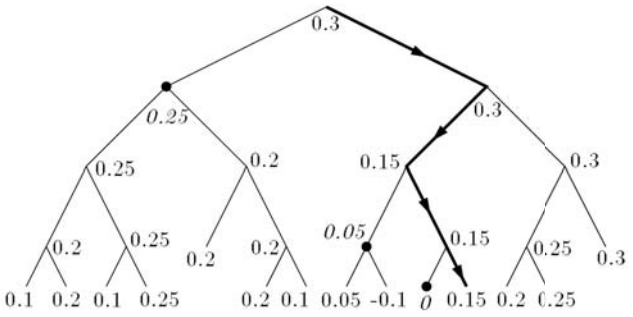


Fig. 4. An illustration of visibility inquiry using search operation on the status-tree. Whenever the search path goes right, it checks the left-child LOS-slope value and compares it against the LOS-slope of the cell we are currently inquiring. If it is higher, it means the cell is not visible [4].

- Creating the event-list
- Sorting the event-list
- Build a histogram for all events based on the angle value (theta) and then apply prefix sum on the histogram
- Divide the event-list between threads and start computing the visibility in parallel

We start the event-list building phase by first transferring the elevation-grid from host to device. The elevation-grid is a 1D array that contains the elevation values for all cells in the grid. We map the index of a certain value in the array to the original cell coordinates using the following equations:

$$Cell_x = index \mod (NumOfColumns) \tag{5}$$

$$Cell_y = \frac{index}{NumOfRows} \tag{6}$$

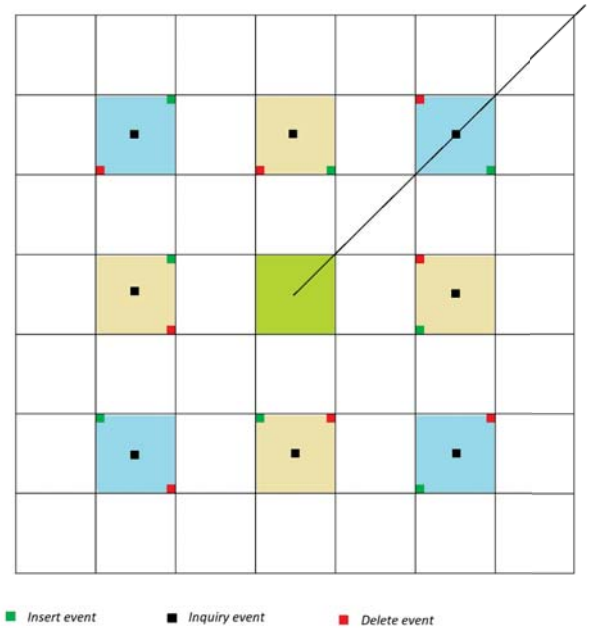


Fig. 5. Different events position based on the cell location. There are 8 different possible locations a cell can be at.

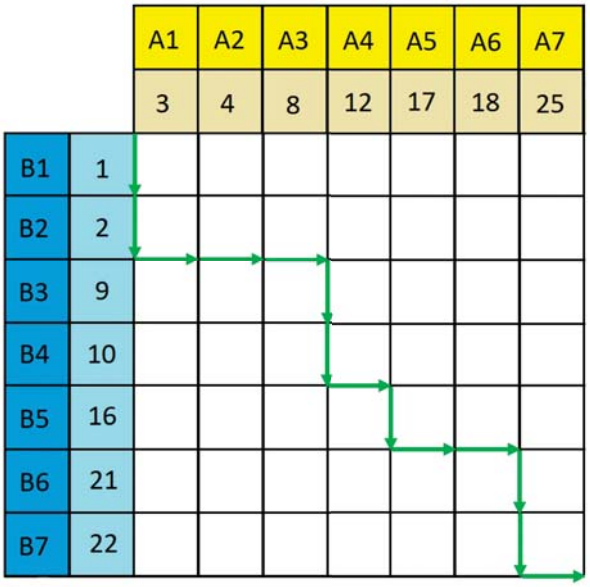


Fig. 6. An example of mergePath algorithm, assume here that each thread merges 7 numbers. The cross diagonal from the above example equals 3. This means thread1 will focus on first 3 elements from array A, and first 7-3 = 4 elements from array B. Thread2 will merge the last 7-3=4 elements from array A, and last 3 elements from array B.

Each cell will be assigned to one thread, which we found it is a little faster than assigning a group of cells to a single thread. Each thread will compute all needed information for a given cell, such as distance to the observer, elevation slope, event type (2: insert, 1: inquiry, 0: delete), and angle values (theta) for all three events. After that, it stores all three events information in the event-list array which is already allocated in the GPU memory.

The second step is sorting event-list on GPU. We used a parallel merge sort called "merge path" on GPU [13]. In order to implement the intuitive merge sort algorithm in a parallel manner, the algorithm applies a merge-path approach, which is a preliminary step before each merge iteration in order to tell each thread how many elements from each sorted-subarray it can take and merge into a fixed-sized sorted chunk. Assume we have two sorted subarrays in the form of [A] x [B] matrix, the merge path is the series of comparison results of the elements from the two subarrays. From Fig. 6, we can see by using the merge-path approach, we can divide the subarrays into independent segments by using what is called cross-diagonals. Each thread will work on a single segment where it combines elements from both segments in a merge fashion. At first, the unsorted array will be divided into equal-sized chunks between blocks. For each block, threads will fetch their chunk of data into the shared memory and then apply sort-merge using a merge-path to sort data stored in shared memory. All chunks will be stored again in global memory, and cooperatively many blocks will participate in sorting and merging the sorted chunks using the merge-path approach in every iteration until data are completely sorted.

However, this approach can sort an array based on a single key attribute, and we need to sort the event-list based on three attributes (location-angle, distance, event-type). In order to do so in a very efficient manner, we converted location-angle (theta) and distance into integer data type while preserving what is after the decimal point (e.g. 90.233 will become 90233), and fortunately we what to sort both fields in ascending order. For the event-type field, we want to delete events to come first then inquiry events, and last insert events. This is when we have events with the same location-angle theta and with the same distance value. Thus as mentioned before, we assign value 0 for delete, 1 for inquiry, and 2 for insert as an event type, and it will be consistent with the ascending order sorting that we want to do for the event-list. As we said before, merge-sort only viable for sorting an array based on only a single key or field. In order to do so we merged all three fields in a single long data type variable (e.g. theta = 180.005, distance = 5000.011, event type = 2, the combined value is 18000550000112) and unsigned long usually can hold a value up to $1.84 * 10^{19}$ which means we can put at least up to 19 digits altogether. Such a trick will benefit in reducing the event-list size to more than half, which is significant.

The third step in our algorithm is to build a histogram for all location-angles and then apply prefix-sum on the histogram. The reason for this is that we want to divide the grid into sectors of equal values of angles. For example, we want to divide event-list into partitions, and each one will have ten or

twenty different angles, and every thread will process only one partition. As we mentioned before, we did convert location-angle from double to integer data type, and this made it easier to build the histogram and then applying prefix-sum on it. Note that in our case, all angles only had three digits after the decimal point, and to make a full sweeping rotation, the sweeping line has to rotate for 360°, so the histogram size should be 360*1000 = 360,000 slots and almost 99.8% of the histogram slots had values other than zero. After that, the prefix-sum operation is applied, so each thread will know its starting point of the partition it has to process in the event-list because the total number of events each thread has to process varies due to the nature in how a 2D grid and its cells arrangement is built. The last step is that each thread will apply the radial sweep approach on its designated partition from the event-list similar to how the sequential radial-sweep when working on the whole event-list. Note, however, each thread will have its own status-tree where it inserts cells into the tree, inquiry them, or delete them. Two essential things worth mentioning in this step. First, when implementing the status-tree in the sequential version of the algorithm building a dynamic binary tree that uses pointers in its build tends to be the right choice, but instead using a pre-allocated static binary tree gives us the following advantages:

- Eliminates the cost of frequently rebalancing the tree with every insert and delete, along with the cost of allocating new nodes and deallocating.
- Significantly reduces the memory space reserved for the status tree now that we do not use pointers where each pointer is used need 4 bytes of space, and we tend to use three-pointers: two for the children and one for the parent.
- Static BST is more suitable to implement on GPU because there is no costly cheap way for allocating and deallocating a space in the global memory to implement a dynamic data structure; thus, a static tree would solve this issue.

The only downfall for using static BST is that it requires to have more nodes than actually needed. For example, in our test case, we need to have 10,000 leaf nodes, so the total number of nodes is twice that much, and the depth of the tree should be log(20,000) + 1= 15 levels. Thus the total number of nodes in a tree with 15 levels = $2^{15} - 1$ so we need to allocate 32767 slots for each status-tree. However, the memory space reduction we gain by eliminating pointers still gives us an advantage over using dynamic BST. In order for each thread to work on its designated partition without any concerns, each thread needs to initialize its status-tree by adding all cells that intersect with the line of angle theta. To do so, we need the line increment values for x and y, which can be obtained using equations (7) and (8).

$$Xstep = \cos\theta \tag{7}$$

$$Ystep - \sin\theta \tag{8}$$

We start increasing x-value and y-value starting from the observer cell coordinates until we reach any of the four end borders of the grid. With every increment (or decrement based on the line direction) we figure out which cell the line is currently crossing, and we re-compute that information for insert event only for that cell and then inserted to the status-tree. After that, each thread will start fetching events information from its designated partition in the event-list and apply the proper operation until it finishes all events in its partition. When handling cells visibility results, each thread will update the visibility value for the designated cell in global memory, and there is no need for an atomic operation to handle the global memory updating because every thread works on different cells and there will not be two threads inquiring the visibility of the same cell.

D. Compressing and reducing Intermediate Size and Cost

In general, one of the downfalls of the radial-sweep algorithm over R3 is that it generates a large amount of intermediate data such as event-list and status-tree. As we showed earlier, using static BST over dynamic BST for implementing the status-tree could save us at least 60% - 70% space. Where the tree-node object contains three-pointers and two integer variables for distance and LOS-slope. We figured out that with static BST the only needed information here is LOS-slope while distance which is the key in determining if a node needs to be added to the left or right, can be estimated by evenly divide the possible longest distance value among the nodes, and by having a static status-tree that is larger than what we need makes it more viable to do so. Thus it ensures that cells that are currently intersected with the sweeping line are evenly distributed within the status-tree. Another space reducing technique we used is to convert all information that is double-precision data type to integer data type while preserving all digits after the decimal point. This would save us half the needed space and would reduce the computation cost when working with integers instead of float or double precision data.

In our experiment, we had an elevation grid that is (10801 x 10801) which results in a little over 116 million cells. The size of the intermediate data is as the following: event-list size was three times the size of the grid, and each event contains five pieces of information: location-angle, distance, LOS-slope, cell-index, and the event-type. The first three variables are double data type and converting those into integer would save us 40% space on memory. The second intermediate data is status-tree, which, as we said converting it to a static BST instead of dynamic, would save us at least 60%, and in our experiment, we saved 80% space. A status-tree would have at most n cells information at any given time, and as we mentioned earlier for our experiment, the overall size of the status-tree was (2^{15}–1) nodes, which are around 0.125 MB

per tree. We mentioned earlier that each thread would need to have its own status-tree. Having to run 5000 threads will require us to reserve 0.61 GB for all the trees, which are not bad at all, and for the event-list, after converting to an integer, the total space needed to be reserved on global memory was 5.87 GB.

IV. EVALUATION

In this section, we evaluated the performance of our GPU-based parallel radial-sweep algorithm and compared it against sequential R3 and Radial-sweep algorithms on the CPU. Also, we compared it with a parallel R3 algorithm on the GPU that we implemented. Also, at the end of our evaluation, we showed the effect of using a static data structure to serve as a "status structure" instead of a dynamic data structure.

A. Experimental Setup and Results

In our experiments, we used Titan V graphic card to execute GPU-based R3 and our radial-sweep algorithm. For sequential viewshed algorithms R3 and radial-sweep, we used AMD Ryzen Threadripper 1950x 16-core 2.15 GHz processor on a machine running Ubuntu 18.04.3 LTS operating system. All CPU and GPU codes are compiled using NVCC 10.

We have used two different datasets in the experiments. The data used in the first experiment is a DEM elevation-grid for Germany obtained from [14]. The grid is composed of (10801 x 10801) cells, and the elevation-grid size on memory is around 0.43 GB. In the second experiment, we have used two grids from SRTM datasets that are used by [5]. The grids dimensions are (5000x5000) and (10,000x10,000). In the first experiment, both CPU-based viewshed algorithms took much longer than expected even though we were assuming the case that data and any intermediate results will fit in memory during the execution. R3 algorithm was the worst in terms of performance, and needed a little less than 3 hours to finish execution while radial-sweep finished execution in 56 minutes. The execution time for R3 on GPU was 6.5 seconds, which makes it 1627x times faster than its CPU-based implementation. Our GPU-based radial-sweep execution time was 1.7 seconds which makes it the fastest among all other viewshed algorithm implementation so far. We gain more than 3.5x speedup over GPU-based R3, 1980x speedup over its CPU sequential version, and 6200x faster than CPU-based R3. See Fig.7.

Fig. 8 illustrates the time break down for our parallel radial-sweep algorithm on GPU. 62% of the execution time goes to the radial-sweep phase, which is the final step in the algorithm and the most time-consuming phase. The second time-consuming step is the sorting step, which makes 23% of the total execution time. Building the histogram step makes 7% of total execution time. Note, however, there is a possibility for further optimization here if shared memory would be used to improve the performance of this operation, but optimizing histogram operation will have negligible improvement compared to the overall performance. The shortest steps in our algorithm are applying prefix-sum on the histogram and event-list building step which is the very first step in the algorithm. The cost of performing prefix-sum on the histogram was minimal (2.5 milliseconds) which is around 0.15%, and building the event-list is 1.5% of overall execution time. Finally, the time required for transferring data to and from the GPU device composes a little over 5% of the total execution time. In other words, 5% of the algorithm execution time is for I/O, while 95% is the actual implementation. See Fig. 9,

The second experiment shows similar performance behavior to the previous one. CPU-based radial-sweep and R3 performed the worst, and the execution time is almost the same as the previous experiment. For GPU-based R3 and radial-sweep, the difference in their execution time is minimal when grid size is small (5000x5000). However, the difference in performance is more obvious where R3 on the GPU finished execution in 5 seconds while GPU-based radial-sweep finished in 1.5 seconds. See Fig. 10.

One of the main components of the parallel radial-sweep algorithm on the GPU is using a static binary search tree instead of a dynamic binary search tree as a status-structure that manages events that are intersecting with sweeping-line. To show the performance improvement of using static over dynamic data structure in radial-sweep algorithm, we designed another variation of the sequential radial-sweep algorithm on the CPU that uses static BST to maintain active events with no other optimizations whatsoever, and we tested on the same dataset used by [5] and also used in the previous experiment in this paper. We were able to test both algorithm variations on large grids up to 25000x25000 cells in size. We choose to experiment on the CPU instead of the GPU because Computing such large grids on the CPU is possible due to that its main memory is much larger than GPU memory, and using dynamic data-structures that are continuously allocating and deallocating is very costly on the GPU and inefficient. Thus, experimenting on the CPU is a more suitable option. Our results show that static BST variation is 10x to 44x faster than the original radial-sweep using dynamic BST. See Fig.11 and Fig.12. Also, the static-variation requires at least 72% less memory for the status-structure. See Fig.13

V. CONCLUSIONS AND FUTURE WORK

In this paper, we proposed a new parallel radial-sweep algorithm designed for GPU. The algorithm outperforms all other algorithms that are defined as "exact" viewshed algorithms. Parallelizing the radial-sweep algorithm for a large number of threads was a challenging task. It requires using many parallel-primitives that have been developed for GPU over the years, such as parallel sort-merge, building histogram, and prefix-sum operation. Along with replacing dynamic data structure used to manage intermediate results with a static data structure. Our experiments showed that the parallel radial-sweep algorithm is up to 3.5x faster than the fastest available viewshed algorithm. Also, we showed that, in general, using a static data structure can boost radial-sweep algorithm performance to up to 44x. However, it is clear that the radial-sweep algorithm's most significant limitation is memory consumption for storing

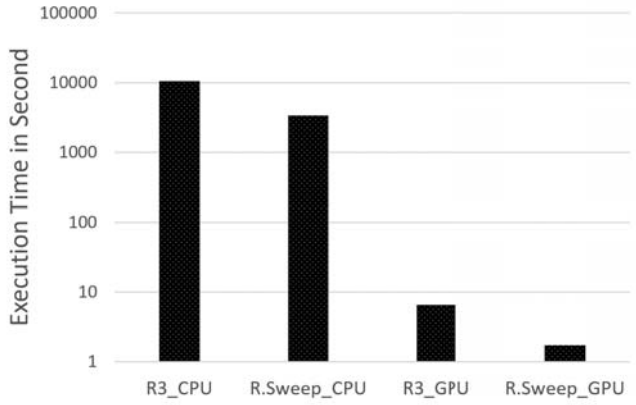


Fig. 7. Experiment 1: Execution time for all four algorithms on a grid size (10801x10801)

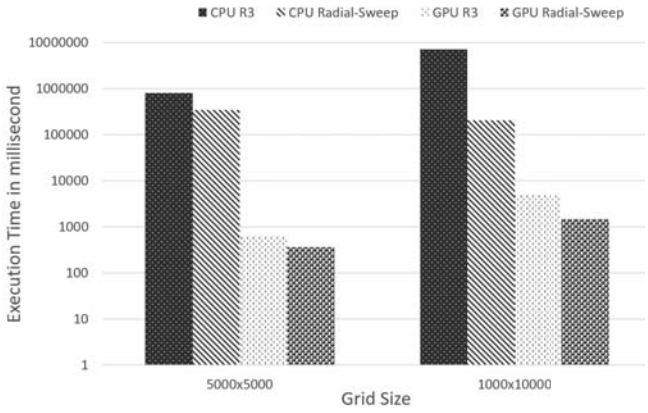


Fig. 10. Experiment 2 results on two different grid sizes (5000x5000) and (10000x10000)

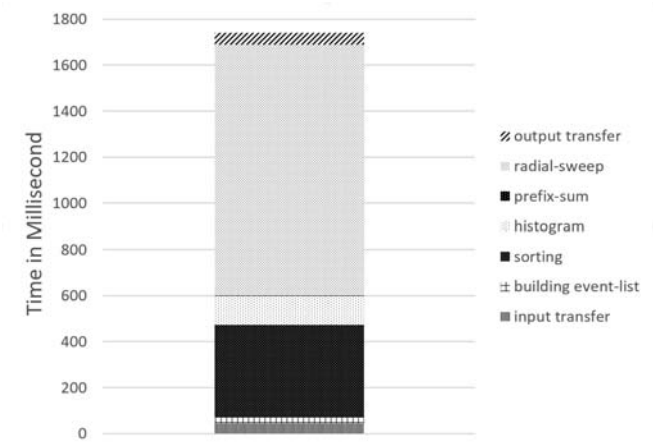


Fig. 8. The break-down of execution time for GPU-based radial-sweep algorithm stages taken from the first experiment

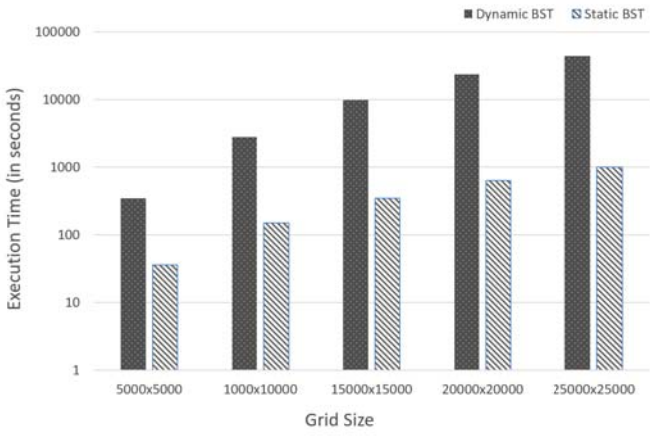


Fig. 11. The execution time for two variations of CPU-based sequential radial-sweep

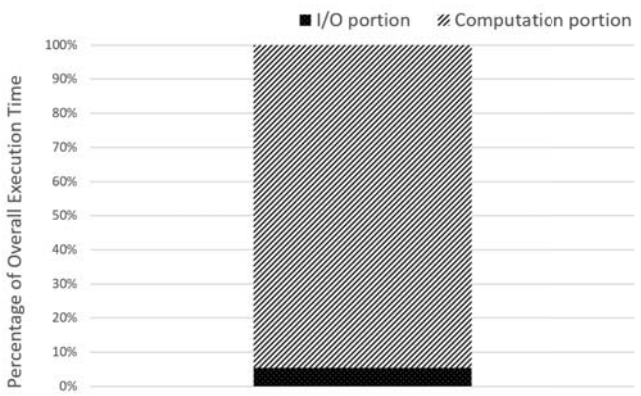


Fig. 9. The break-down percentage of the I/O portion and the computation portion of the overall execution for parallel radial-sweep on GPU

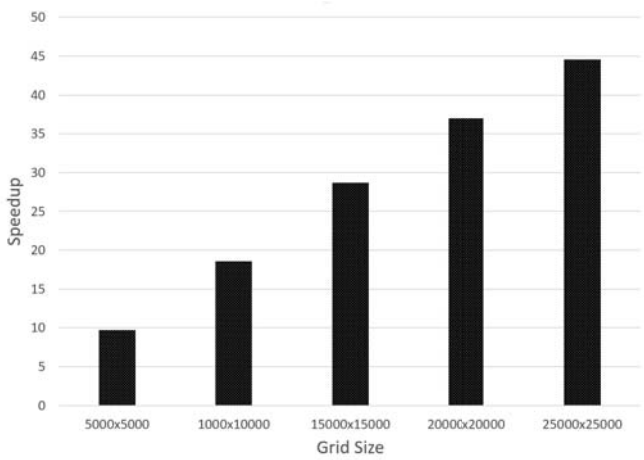


Fig. 12. The performance speedup of CPU-based sequential radial-sweep using static BST over the original radial-sweep that uses dynamic BST as a status-structure

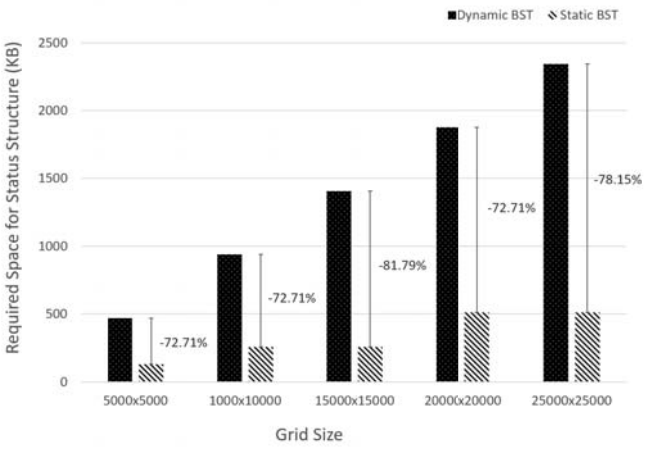


Fig. 13. Memory requirement for status structure for two variations of CPU-based sequential radial-sweep. The first variation uses dynamic BST and the second uses static BST

intermediate results generated by the algorithm.

Our research can be further extended to include the use of multiple GPUs when data are enormous, or by exploiting an efficient I/O optimization technique such as GPUs data stream system along with pinning slots on the CPU RAM to reduce the overhead of the frequent transfers between host and device to overcome the limitations of the radial-sweep algorithm.

REFERENCES

[1] L. Floriani and P. Magillo, "Algorithms for visibility computation on terrains: a survey," *Environment and Planning B: Planning and Design*, vol. 30, no. 5, pp. 709–728, 2003.

[2] A. Hrovat, I. Ozimek, A. Vilhar, T. Celcer, I. Saje, and T. Javornik, "An open-source radio coverage prediction tool," in *14th WSEAS International Conference on Communications, Corfu Island, Greece*, 2010.

[3] W. R. Franklin and C. Ray, "Higher isn't necessarily better: Visibility algorithms and experiments," in *Advances in GIS research: sixth international symposium on spatial data handling*, vol. 2. Taylor & Francis Edinburgh, 1994, pp. 751–770.

[4] M. Van Kreveld, *Variations on sweep algorithms: efficient computation of extended viewsheds and class intervals*. Utrecht University: Information and Computing Sciences, 1996, vol. 1996.

[5] C. R. Ferreira, M. V. A. Andrade, S. V. Magalhães, W. R. Franklin, and G. C. Pena, "A parallel algorithm for viewshed computation on grid terrains." *JIDM*, vol. 5, no. 2, pp. 171–180, 2014.

[6] H. Haverkort, L. Toma, and Y. Zhuang, "Computing visibility on terrains in external memory," *Journal of Experimental Algorithmics (JEA)*, vol. 13, p. 5, 2009.

[7] Y. Zhao, A. Padmanabhan, and S. Wang, "A parallel computing approach to viewshed analysis of large terrain data using graphics processing units," *International Journal of Geographical Information Science*, vol. 27, no. 2, pp. 363–384, 2013.

[8] A. Shapira, "Visibility and terrain labeling," *Master's thesis, Rensselaer Polytechnic Institute*, 1990.

[9] A. Osterman, L. Benedičič, and P. Ritoša, "An io-efficient parallel implementation of an r2 viewshed algorithm for large terrain maps on a cuda gpu," *International Journal of Geographical Information Science*, vol. 28, no. 11, pp. 2304–2327, 2014.

[10] C. R. Ferreira, S. V. Magalhães, M. V. Andrade, W. R. Franklin, and A. M. Pompermayer, "More efficient terrain viewshed computation on massive datasets using external memory," in *Proceedings of the 20th international conference on advances in geographic information systems*. ACM, 2012, pp. 494–497.

[11] N. Stojanović and D. Stojanović, "Performance improvement of viewshed analysis using gpu," in *2013 11th International Conference on Telecommunications in Modern Satellite, Cable and Broadcasting Services (TELSIKS)*, vol. 2. IEEE, 2013, pp. 397–400.

[12] F. V. Maciorov, *Electronic digital integrating computers: digital differential analyzers*. Iliffe Books, 1964.

[13] O. Green, R. McColl, and D. A. Bader, "Gpu merge path: a gpu merging algorithm," in *Proceedings of the 26th ACM international conference on Supercomputing*. ACM, 2012, pp. 331–340.

[14] https://www.opendem.info/download_srtm.html.